

Conferencia 5: cadenas y archivos

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-10-14

Llevas cuatro clases usando cadenas de texto, pero solo para imprimirlas y para leerlas de la consola. Hoy las vamos a abrir por dentro y vas a ver que una cadena es una secuencia, igual que una lista, con las mismas reglas de índices y rebanadas que aprendiste la semana pasada. Y como el texto de verdad no se escribe a mano sino que vive en archivos, hoy también aprendemos a leerlos. El proyecto de la clase es un programa que abre un archivo y cuenta sus palabras.

Trabajamos sobre el archivo `quijote.txt`, que tienes junto a estas notas y contiene los tres primeros párrafos del Quijote.

1. Una cadena es una secuencia

Todo lo que aprendiste de listas sirve aquí. Una cadena tiene longitud, se indexa desde cero, admite índices negativos y se rebaná:

```
palabra = "computación"

print(len(palabra))
print(palabra[0])
print(palabra[-1])
print(palabra[0:5])
print(palabra[5:])
print(palabra[::-1])
```

```
11
c
n
compu
tación
nóicatuPmOC
```

Y se recorre con `for`, elemento por elemento, que en una cadena significa carácter por carácter:

```
for letra in "MatCom":
    print(letra, end="-")
```

```
M-a-t-C-o-m-
```

Contar con esto es inmediato. Cuántas vocales tiene una palabra:

```
palabra = "computación"
vocales = 0

for letra in palabra:
    if letra in "aeiouáéíóú":
        vocales += 1

print(vocales)
```

Ahí hay un detalle que conviene mirar. En una lista, `x in lista` pregunta si `x` es uno de los elementos. En una cadena, `in` pregunta si aparece como **pedazo**, no solo como carácter suelto:

```
print("a" in "hola")
print("ola" in "hola")
print("alo" in "hola")
```

```
True
True
False
```

Eso es cómodo para buscar palabras dentro de un texto y no tiene equivalente en las listas.

Las cadenas no se pueden modificar

Aquí está la diferencia grande con las listas. Una lista se modifica en el lugar; una cadena no:

```
palabra = "hola"
palabra[0] = "H"
```

```
Traceback (most recent call last):
  File "<cell>", line 2, in <module>
    palabra[0] = "H"
    ~~~~~^~~~
TypeError: 'str' object does not support item assignment
```

Las cadenas son **inmutables**, como los números. La consecuencia práctica es que toda operación sobre una cadena construye una cadena nueva y deja la original intacta. Para "cambiar" una cadena hay que reasignar el nombre:

```
palabra = "hola"
mayuscula = palabra.upper()

print(palabra)
print(mayuscula)

palabra = palabra.upper()
print(palabra)
```

```
hola
HOLA
HOLA
```

Y como no se pueden modificar, tampoco tienen el problema de los nombres compartidos que vimos la clase pasada. Dos nombres para la misma cadena nunca se sorprenden el uno al otro, porque ninguno puede cambiarla.

2. Métodos de cadena

Una cadena trae muchas funciones propias, que se llaman con un punto detrás del valor. Estas son las que vas a usar todo el tiempo:

MÉTODO	QUÉ DEVUELVE
<code>s.lower()</code> , <code>s.upper()</code>	la misma cadena en minúsculas o mayúsculas
<code>s.strip()</code>	la cadena sin espacios ni saltos al principio y al final
<code>s.replace(a, b)</code>	una copia con cada a cambiado por b
<code>s.split()</code>	la lista de las palabras, separando por espacios
<code>s.split(c)</code>	la lista de los pedazos, separando por c
<code>separador.join(lista)</code>	pega los elementos de la lista con ese separador
<code>s.startswith(t)</code> , <code>s.endswith(t)</code>	si empieza o termina con t
<code>s.find(t)</code>	la posición de t, o -1 si no está
<code>s.count(t)</code>	cuántas veces aparece t

Ninguno modifica la cadena: todos devuelven una nueva.

```
línea = "  En un lugar de la Mancha  "

print(repr(línea.strip()))
print(línea.strip().lower())
print(línea.count("a"))
print(línea.find("lugar"))
```

```
'En un lugar de la Mancha'
en un lugar de la mancha
4
8
```

`repr` imprime la cadena con sus comillas, y sirve para ver exactamente dónde están los espacios. Úsalo cuando algo no cuadre.

`split` y `join`

De todos, el importante para hoy es `split`, porque convierte un texto en una lista de palabras y te devuelve al terreno de la clase pasada:

```
línea = "En un lugar de la Mancha"
palabras = línea.split()

print(palabras)
print(len(palabras))
print(palabras[2])
```

```
['En', 'un', 'lugar', 'de', 'la', 'Mancha']  
6  
lugar
```

join hace el camino inverso: recibe una lista y la pega en una sola cadena. Se escribe al revés de como uno esperaría, con el separador delante:

```
palabras = ["En", "un", "lugar"]  
  
print(" ".join(palabras))  
print("-".join(palabras))  
print("".join(palabras))
```

```
En un lugar  
En-un-lugar  
Enunlugar
```

La pareja split y join es la manera normal de transformar un texto en Python: partir en palabras, trabajar con la lista, volver a pegar.

3. Archivos

Un archivo se abre con open, que recibe la ruta y el modo. Los modos son "r" para leer, "w" para escribir desde cero y "a" para agregar al final.

La forma correcta de abrir un archivo es con with, que lo cierra solo al terminar el bloque, incluso si algo falla dentro:

```
with open("quijote.txt", encoding="utf-8") as archivo:  
    contenido = archivo.read()  
  
print(len(contenido))  
print(contenido[:60])
```

```
1101  
En un lugar de la Mancha, de cuyo nombre no quiero acordarme
```

read devuelve todo el archivo como una sola cadena. El with se lee así: abre el archivo, llámalo archivo mientras dure este bloque, y ciérralo al salir. Fuera del bloque el archivo ya está cerrado, pero contenido sigue disponible porque es una variable normal.

La codificación

Ese encoding="utf-8" no es decoración. Un archivo de texto es en realidad una secuencia de bytes, y la codificación es la tabla que dice qué letra representa cada byte. UTF-8 es el estándar de hoy y el que debes usar siempre al crear archivos.

El problema es que todavía circulan archivos viejos en otras codificaciones. Si abres un archivo latino con la codificación equivocada, las tildes y las eñes salen rotas o el programa se cae. Cuando te pase, ya sabes qué buscar. Escribe encoding="utf-8" siempre, explícitamente, aunque tu sistema lo tenga por defecto, porque el programa tiene que funcionar también en la computadora del profesor.

Leer línea por línea

Un archivo se puede recorrer con for como cualquier secuencia, y entonces cada elemento es una línea. Esto es mejor que read para archivos grandes, porque no carga todo en memoria de una vez:

```
with open("quijote.txt", encoding="utf-8") as archivo:
    for numero, linea in enumerate(archivo, start=1):
        if numero > 3:
            break
        print(numero, repr(linea[:28]), "...", repr(linea[-4:]))
```

```
1 'En un lugar de la Mancha, de' ... 'que\n'
2 'vivía un hidalgo de los de l' ... 'lgo\n'
3 'corredor. Una olla de algo m' ... 's y\n'
```

Fíjate en el final de cada línea: leer una línea incluye el \n del salto. Por eso casi siempre se le hace strip() antes de trabajar con ella.

Escribir

Con el modo "w" se escribe. Cuidado, porque **borra el archivo si ya existía**:

```
with open("salida.txt", "w", encoding="utf-8") as archivo:
    archivo.write("primera línea\n")
    archivo.write("segunda línea\n")

with open("salida.txt", encoding="utf-8") as archivo:
    print(archivo.read())
```

```
primera línea
segunda línea
```

write no agrega el salto de línea; hay que ponerlo a mano.

4. Cuando las cosas salen mal

Todo lo anterior supone que el archivo existe. Si no existe, el programa se muere:

```
with open("no-existe.txt", encoding="utf-8") as archivo:
    print(archivo.read())
```

```
Traceback (most recent call last):
  File "<cell>", line 1, in <module>
    with open("no-existe.txt", encoding="utf-8") as archivo:
        ~~~~~
FileNotFoundError: [Errno 2] No such file or directory: 'no-existe.txt'
```

Hasta hoy, cada error que hemos visto ha terminado el programa. Un programa serio no puede darse ese lujo: si el usuario escribe mal el nombre del archivo, lo correcto es decírselo y seguir, no volcarle un traceback en la pantalla.

Para eso está try. El bloque try contiene lo que puede fallar, y el except dice qué hacer si falla:

```
try:
    with open("no-existe.txt", encoding="utf-8") as archivo:
        contenido = archivo.read()
    print(f"El archivo tiene {len(contenido)} caracteres")
except FileNotFoundError:
    print("No encontré ese archivo")

print("y el programa sigue")
```

```
No encontré ese archivo
y el programa sigue
```

Esa última línea es el punto entero de la sección. El error ocurrió, el programa lo atendió y continuó.

FileNotFoundError es el nombre del tipo de error, el mismo que aparecía en la última línea del traceback. Cada tipo de problema tiene el suyo, y ya te has encontrado varios este curso:

ERROR	CUÁNDO
ValueError	int("hola"), el texto no representa un número
IndexError	lista[10] en una lista de tres elementos
NameError	usar una variable que no existe
ZeroDivisionError	dividir entre cero
FileNotFoundError	abrir un archivo que no está
TypeError	"5" + 3, operar tipos incompatibles

Una función que atiende su propio error queda así. Un try puede llevar varios except, uno por cada tipo de problema, y se ejecuta el primero que empareja:

```
def leer_entero(texto):
    """Convierte texto a entero. Devuelve None si no se puede."""
    try:
        return int(texto)
    except ValueError:
        print(f"'{texto}' no es un número entero")
        return None

print(leer_entero("42"))
print(leer_entero("3.5"))
print(leer_entero("hola"))
```

```
42
'3.5' no es un número entero
None
'hola' no es un número entero
None
```

Una advertencia. Se puede escribir except: a secas, sin nombrar el error, y atrapa cualquier cosa. **No lo hagas.** Atrapa también los errores que no esperabas, incluidos tus propios fallos de programación, y los esconde. Nombra siempre el error que sabes manejar y deja que los demás se vean.

5. El proyecto: contar las palabras

Ya están todas las piezas. El programa abre un archivo y reporta cuántas palabras tiene, cuál es la más larga y cuál es el promedio de letras.

El primer problema es que split a secas no sirve: deja los signos de puntuación pegados, así que Mancha, y Mancha serían palabras distintas. Hay que limpiar antes de partir.

```
SIGNOS = ".,:;¿?¡!()«»\\"'--"

def limpiar(texto):
    """Devuelve el texto en minúsculas y sin signos de puntuación."""
    texto = texto.lower()

    for signo in SIGNOS:
        texto = texto.replace(signo, " ")

    return texto

print(limpiar("En un lugar de la Mancha, ¿de cuyo nombre...!"))
```

```
en un lugar de la mancha  de cuyo nombre
```

SIGNOS va en mayúsculas por convención: es una constante, un valor que se fija una vez y no cambia. Python no lo impide, pero el nombre en mayúsculas avisa al que lea.

Con eso, el resto sale corto:

```
def palabras_de(ruta):
    """Devuelve la lista de palabras de un archivo de texto."""
    with open(ruta, encoding="utf-8") as archivo:
        return limpiar(archivo.read()).split()

def mas_larga(palabras):
    """Devuelve la palabra más larga de la lista."""
    mayor = palabras[0]

    for palabra in palabras:
        if len(palabra) > len(mayor):
            mayor = palabra

    return mayor

def promedio_de_letras(palabras):
    """Devuelve el promedio de letras por palabra."""
    letras = 0

    for palabra in palabras:
        letras += len(palabra)

    return letras / len(palabras)

palabras = palabras_de("quijote.txt")

print(f"{len(palabras)} palabras")
print(f"la más larga es '{mas_larga(palabras)}'")
print(f"promedio de {promedio_de_letras(palabras):.2f} letras")
```

```
196 palabras
la más larga es 'entresemana'
promedio de 4.46 letras
```

`mas_larga` es el mismo acumulador de la clase pasada, con `len(palabra)` en lugar del valor. Ese patrón, guardar el mejor visto hasta ahora, lo vas a escribir cien veces.

Y el programa principal, con la ruta pedida al usuario y el error atendido:

```
ruta = input("Archivo: ")

try:
    palabras = palabras_de(ruta)
except FileNotFoundError:
    print(f"No encontré el archivo {ruta}")
else:
    print(f"{len(palabras)} palabras")
    print(f"la más larga es '{mas_larga(palabras)}'")
```

El `else` de un `try` se ejecuta solo si no hubo error, y sirve para dejar dentro del `try` únicamente la línea que puede fallar. Cuanto más pequeño el `try`, más claro qué error estás atendiendo.

Queda una pregunta natural que hoy no podemos responder: **cuántas veces aparece cada palabra**. Podrías intentarlo con dos listas en paralelo, una de palabras y otra de conteos, y funcionaría, pero sería lento y incómodo. La estructura que resuelve eso es el diccionario, y es la clase que viene.

6. Resumen

- Una cadena es una secuencia: `len`, índices desde cero, índices negativos, rebanadas y `for`, igual que una lista.
- En una cadena, `in` busca un pedazo, no solo un carácter.
- Las cadenas son inmutables. Todo método devuelve una cadena nueva y deja la original intacta.
- `split` convierte texto en lista de palabras; `join` hace lo contrario.
- Los archivos se abren con `with open(ruta, encoding="utf-8")`, que los cierra solo.
- El modo "w" borra lo que había. Recorrer el archivo con `for` da una línea por vuelta, con su salto de línea incluido.
- `try` y `except` permiten que un programa atienda un error en vez de morirse. Nombra siempre el error concreto; `except:` a secas esconde tus propios fallos.

Ejercicios

1. Escribe `es_palindromo(frase)` que diga si una frase se lee igual al derecho y al revés, ignorando mayúsculas, espacios y signos. Debe dar `True` con "Dábele arroz a la zorra el abad".
2. Escribe `contar_lineas(ruta)`, `contar_palabras(ruta)` y `contar_caracteres(ruta)`. Compara tus resultados con los del comando `wc` de la terminal sobre el mismo archivo, y explica cualquier diferencia.
3. Escribe `capitalizar(frase)` que ponga en mayúscula la primera letra de cada palabra, sin usar el método `title` ni `capitalize`. Usa `split`, un ciclo y `join`.
4. Escribe un programa que lea `quijote.txt` y escriba `quijote-numerado.txt` con el mismo contenido pero con cada línea precedida de su número y un espacio.

5. Escribe `palabras_largas(ruta, minimo=8)` que devuelva la lista de las palabras distintas del archivo con al menos `minimo` letras, sin repeticiones y en orden alfabético. Puedes usar `sorted`.
6. Escribe un programa que le pida números al usuario uno por línea hasta una línea vacía, y que sume solo los que sean válidos, avisando de cada entrada mala sin terminar. Debe sobrevivir a que el usuario escriba `hola`, `3.5` y `--`.
7. Sin ejecutarlos, di qué imprime cada uno de estos tres programas. Después compruébalo.

(a)

```
s = "hola"
s.upper()
print(s)
```

(b)

```
print(len("a,b,,c".split(",")))
print(len("a b c".split()))
```

(c)

```
try:
    print(int("7") + int("ocho"))
except ValueError:
    print("mal")
except TypeError:
    print("peor")
```